



University at Buffalo SEDS

RFBoard26

Software Design Document

September 11, 2026

Expanded implementation baseline

Version 1.1 — LaTeX / Overleaf edition

Architecture, drivers, execution, interfaces, and verification

Contents

1 Purpose and scope	1
2 System context and architecture	1
3 Required drivers and platform services	2
4 Tasks, ownership, and synchronization	3
4.1 Application tasks	3
4.2 Data ownership and execution boundaries	4
5 Control flow and operating scenarios	4
5.1 Boot sequence	4
5.2 Telemetry service pass	4
5.3 Ground command and returned state	5
5.4 GPS acquisition and time	6
5.5 Managed-variable synchronization	6
6 Network inputs, outputs, and contracts	6
6.1 Router contract	6
6.2 What arrives and what RFBoard sends back	6
6.3 UART framing and backpressure	7
6.4 CAN packet transport	7
7 Persistent and volatile data	8
7.1 Flash layout	8
7.2 Durable application values	8
7.3 Volatile state	9
8 OTA update design	9
9 Resource and timing budgets	10
10 Fault handling and trust boundaries	11
11 Build, deployment, and verification	11
11.1 Reproducible configuration	11
11.2 Design-to-test traceability	12
12 Open engineering work	12
13 Design rationale and references	13
14 Detailed execution and driver design	14
14.1 Interrupt-to-task handoff	14
14.2 Radio receive algorithm	15
14.3 Radio transmit lifecycle	15
14.4 CAN fragmentation and reassembly	16
14.5 GPS parsing, freshness, and receiver recovery	16
15 Detailed data consistency and state design	17
15.1 Lifecycle of a managed value	17
15.2 Two-bank persistence algorithm	17
15.3 Proposed persistence retry contract	18
15.4 Clock representation and consistency	18

16 Interface contracts and integration examples	19
16.1 API responsibilities	19
16.2 Example: valid underglow update	19
16.3 Example: OTA chunk progression	19
16.4 Discovery, reconnect, and stale traffic	20
17 Qualification scenarios and acceptance planning	20
17.1 Startup and synchronization scenarios	20
17.2 Parser and transport scenarios	20
17.3 State, flash, and update scenarios	21
17.4 Performance and soak evidence	21
18 Maintenance and implementation guidance	21
18.1 Change-impact checklist	21
18.2 Recommended implementation sequence	22
A Network schema catalog	23
A.1 Endpoints	23
A.2 Data types	24
A.3 Schema descriptions	25
A.4 Separate protocol contracts	27

List of Figures

1	System context and software component interactions	2
2	Application startup and concurrent task launch	5
3	Ground command and destination acknowledgement sequence	5
4	OTA stream lifecycle	10
5	Interrupt-to-task receive handoff	14
6	UART frame parsing and resynchronization	15
7	Managed-value lifecycle and proposed persistence retry	17
8	Two-bank persistent-store append and compaction flow	18

Document control	Value
Version	1.1 — expanded LaTeX implementation baseline
Prepared	2026-09-11
Source baseline	RFBoard26, commit b4eba479b57f767d07c7c9a811c741bc043990e6
Target	STM32G491, ThreadX, SEDSNet, SEDS LaunchCore
Audience	Firmware developers, avionics integration engineers, ground-software developers, reviewers
Evidence	Board source and inspected working-tree schema (see Appendix A note), build configuration, locally fetched dependency source, and existing tests
Status	Describes inspected implementation; proposed improvements are explicitly identified

1 Purpose and scope

RFBoard26 connects the avionics CAN-FD network to the RFD900x radio link. It forwards ground commands toward avionics boards and returns their telemetry and responses to the ground. It also acquires NEO-M9N GPS data, supplies GPS-backed network time, applies avionics-bay underglow settings, caches flight state, and accepts firmware delta updates.

The primary design objectives inferred from the implementation are:

ID	Objective	Implementation allocation
D-01	Route traffic in both directions without manually duplicating packets	SEDSNet router with separate CAN and radio sides
D-02	Publish GPS position and satellite information at a bounded rate	GPS task and compile-time telemetry period
D-03	Distribute usable UTC time when GPS date/time is valid	GPS parsing, local epoch update, SEDSNet time synchronization
D-04	Preserve selected local state across reset	LaunchCore persistent key/value storage
D-05	Support bounded firmware update reception	P2P OTA stream and 24 KiB delta partition
D-06	Keep routing responsive under load and backpressure	Bounded service passes, fixed queues, isolated allocator pools
D-07	Expose faults and qualify the complete communications path	Diagnostic counters, stack checks, host tests, linked simulation

These are document-local derived objectives, not formally approved mission requirements. This repository does not establish actuator command semantics, flight-state names/transitions, a mission safety case, or hard end-to-end latency requirements.

RFBoard is not the flight controller, valve controller, or telemetry recorder. Schema endpoints such as SD_CARD identify destinations elsewhere in the network; their presence does not imply local SD-card hardware. Similarly, IMU, pressure, load-cell, and battery message definitions do not imply those sensors are attached to RFBoard.

2 System context and architecture

The application is predominantly C11. ST HAL/CMSIS provide MCU support; ThreadX supplies scheduling, mutexes, and byte pools; SEDSNet is a fetched Rust library accessed through C bindings. LaunchCore supplies boot selection, image handling, updates, and persistence. USBX provides optional debug CDC output.

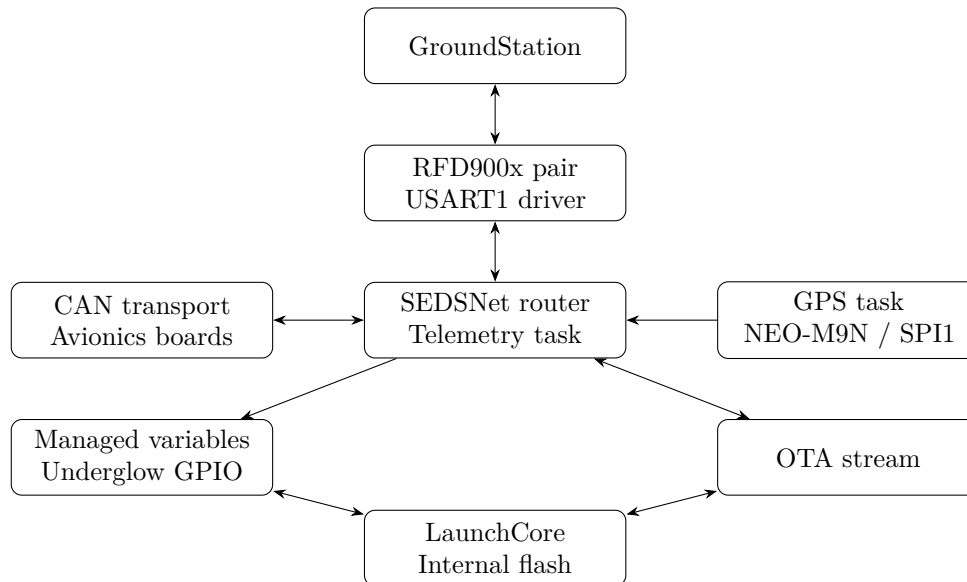


Figure 1: System context and software component interactions

Component	Responsibility	Principal source
Board startup	Clocks, pins, peripherals, pre-network state restoration	main.c
RTOS composition	Pools, telemetry/GPS thread creation, stack-error hook	app_threadx.c , Azure RTOS initialization
Network service	Router lifecycle, side callbacks, discovery, time sync, CAN retry	telemetry.c , telemetry_thread.c
Rust platform integration	Allocator arenas, network mutex, diagnostics/panic hooks	telemetry_hooks.c
Physical transports	UART framing/DMA and CAN framing/reassembly	radio.c , can_bus.c
GPS	Receiver setup, sentence parsing, position/time publication	neom9n_thread.c , neom9n.c , neom9n_config.c
Local state	Underglow output and last-known flight-state cache	av_bay_underglow.c , flight_state_cache.c
Storage/update	Serialized key/value access and OTA command handling	persistent_store.c , ota_stream.c
Boot platform	Flash access/layout, reset and vector validation	Bootloader

3 Required drivers and platform services

The following drivers already exist and are required to reproduce this firmware. New work should extend these interfaces rather than add duplicate owners of the same peripheral.

Driver/service	Hardware/configuration	Required behavior and caller
Clock, GPIO, interrupt controller	170 MHz system clock; CubeMX-generated configuration	Establish clocks and safe initial GPIO levels before scheduler start
FDCAN2 HAL + board transport	PB12 RX, PB13 TX; approximately 1 Mbit/s; FD without bit-rate switching; hardware automatic retransmission	Accept CAN frames, defer reassembly to telemetry task, fragment outgoing packets, return bounded errors under congestion
USART1 + RX/TX DMA	PA9 TX, PA10 RX; 57,600 baud, 8N1; hardware flow control disabled	Receive-to-idle buffering, framed serial parser, owned TX FIFO, DMA completion/error recovery
SPI1 + GPS chip select	PA5 SCK, PA6 MISO, PA7 MOSI; PB1 CS; mode 0, 8-bit, MSB first, prescaler 16	GPS task is the peripheral owner; send receiver configuration and acquire NMEA bytes

Driver/service	Hardware/configuration	Required behavior and caller
NEO-M9N protocol driver	GGA and RMC sentences; configuration applied to receiver RAM at boot	Parse position, fix, satellites, UTC date/time; report timeout, SPI, parse, and overflow errors
Internal flash HAL + LaunchCore adapter	512 KiB internal flash; 2 KiB erase units; 8-byte write alignment	Range checking, bank/page selection, erase/program/read, readback verification; used by bootloader, persistence, and OTA
GPIO output	PB15 blue LEDs/underglow; PB11 green status LED	Restore/apply underglow; indicate boot, activity, and fatal states
ThreadX time base	1,000 ticks/s	Scheduling and local monotonic timing; HAL millisecond tick is maintained through TIM6
USB FS PCD + USBX CDC ACM	Enabled by default for Debug; disabled for Release	Best-effort printf output to an attached/configured host with DTR asserted
Cortex-M startup/fault support	Vector table at 0x08004200	Reset entry, interrupt dispatch, fault capture, stack overflow handling

Pin and peripheral evidence: [main.h](#), [HAL MSP](#), [CubeMX configuration](#). No local ADC, IMU, barometer, SDIO, Ethernet, or IP socket driver is required by the inspected application. The SEDSNet profile name IPV6_LIKE does not establish that this board runs IPv6 sockets.

4 Tasks, ownership, and synchronization

4.1 Application tasks

ThreadX uses lower numerical priorities for higher-priority threads. Both main application threads use matching preemption thresholds, no time slice, and automatic start.

Task/service	Priority	Stack	Trigger/cadence	Work
Telemetry thread	3	14 KiB, allocated from application pool	Repeated service pass followed by 1 tick sleep	CAN startup, router initialization, ingress, routing, discovery/time sync, managed-variable refresh, OTA polling, UART egress
NEO-M9N thread	4	12 KiB static	Initial 5 s delay; acquisition loop followed by 50 ticks sleep	GPS configuration, NMEA reception, fix tracking, satellite/position publication, UTC updates, bounded diagnostic reporting and SPI recovery
USBX device app thread	10	2 KiB from USB pool	Debug startup	Configures USB packet memory, initializes device controller, starts PCD, then returns
ThreadX/USBX internals	Middleware-defined	Middleware-defined	Timers, interrupts, USB events	Kernel scheduling/timers and debug USB class operation

There is no separate application radio task, CAN task, flash task, command task, or OTA task. These

are services run by the telemetry thread or synchronous callbacks. The USB application thread is startup work, not a periodic health-monitor task.

4.2 Data ownership and execution boundaries

Shared object/path	Producer → consumer	Ownership/synchronization contract
UART RX chunks	DMA/interrupt → telemetry task	Copy into bounded RX ring; parser and SEDSNet delivery run in thread context
CAN RX frames	CAN interrupt/hardware FIFO → telemetry task	Buffered frames are reassembled before subscriber delivery
Router state	Telemetry service and GPS publisher	SEDSNet C/Rust integration uses priority-inheritance telemetry mutex hooks; no blocking router work in ISR context
UART TX bytes	Router send callback → DMA	Transport takes an owned copy; retains frame storage through DMA completion; queue mutex protects updates
Pending CAN packets	Send failure → telemetry retry service	Three copied packet slots; retry retains head until successful send
Persistent key/value state	Early startup and variable callbacks	Startup is single-threaded; later calls use priority-inheritance persistence mutex
GPS time	GPS task → time service	GPS time helpers and SEDSNet time API mediate updates; local timing and UTC are separate concepts
Health counters	Drivers/tasks → debugger/simulator	Volatile observations, not an inter-task command channel or a durable event log

Callbacks must not wait for a network reply on the telemetry thread: the same thread must continue servicing ingress to receive that reply. Driver callbacks receive borrowed data; work deferred beyond the callback must use a copy or a dependency-owned object. Flash updates currently execute synchronously within this service path and can extend a pass.

5 Control flow and operating scenarios

5.1 Boot sequence

LaunchCore’s board hook validates vector alignment, initial stack-pointer range, and reset-handler range/Thumb bit. The bootloader occupies a separate flash region. Its image-selection/install policy is implemented in the fetched LaunchCore dependency; this document does not infer a dual-full-image rollback guarantee from the storage API’s `slot_b` name.

Cached underglow is physically applied before ThreadX and network synchronization. The flight-state cache is restored at the same stage but is not itself a flight-state controller. Its weak application hook is a no-op in this repository.

5.2 Telemetry service pass

The actual call order in `telemetry_thread.c` is:

1. Sample stack margin and optionally print health.
2. Process bounded radio RX, then CAN RX/reassembly.
3. Retry pending CAN packets.
4. Poll SEDSNet discovery; poll underglow refresh after successful discovery polling.
5. Retry pending CAN packets again.
6. Process router RX and TX queues with a requested 1 ms service budget.

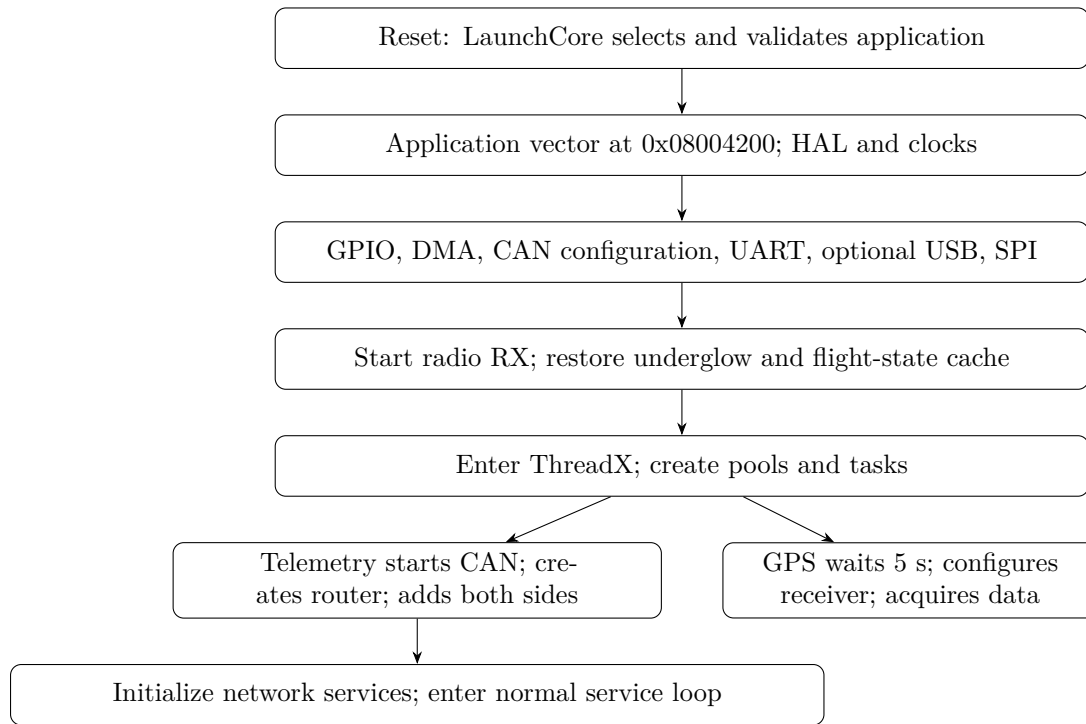


Figure 2: Application startup and concurrent task launch

7. Poll time synchronization; this also polls flight-state refresh.
8. Poll OTA boot confirmation and deferred reset.
9. Service radio TX.
10. Increment loop-completion counter and sleep 1 ms.

The requested queue budget is not a guaranteed 1 ms bound on the whole loop: transport calls, flash operations, callbacks, and scheduling add time. Failed router construction is generally rate-limited to a 5 s retry; some early failure exits do not set the retry deadline.

5.3 Ground command and returned state

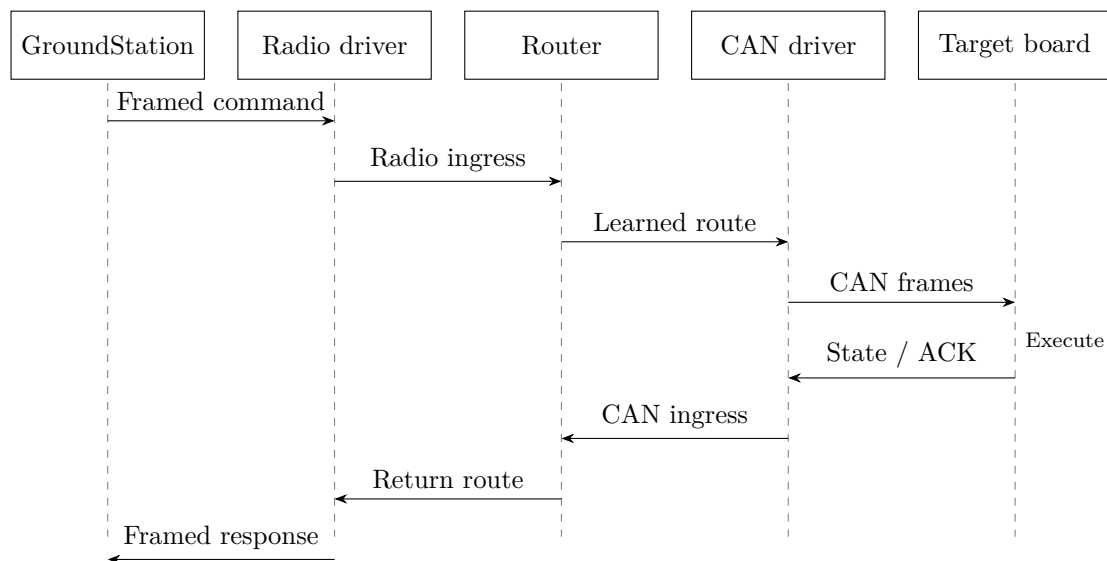


Figure 3: Ground command and destination acknowledgement sequence

RFBoard does not execute a VALVE_COMMAND, ACTUATOR_COMMAND, or FLIGHT_COMMAND. It cannot claim execution success just because its send callback returned success. An actuator state acknowledgement, such as UMBILICAL_STATUS, originates at the owning board; the complete

command/response correlation belongs to the system protocol and GroundStation.

5.4 GPS acquisition and time

After a 5 s receiver boot delay, `gps_config()` requests GGA/RMC output and disables other configured sentence types using RAM-only receiver settings. The acquisition loop parses sentences and publishes position only when a fix is usable. Position is `[latitude_deg, longitude_deg, altitude_msl_m]` as three float32 values. Satellite count is a separate uint8 publication with its own due-time tracking.

Valid GPS date/time is converted to Unix epoch milliseconds and supplied to SEDSNet before production position publication. A separate offset helper supports GPS time-of-day; it must not be confused with epoch time. The configured time-source priority is 1, with 2 s announcement/request intervals and a 5 s source timeout. GroundStation is preferred as discovery master, which is distinct from selection of a time source.

The default telemetry rate is 1 Hz; the accepted compile-time range is 1–1000 whole Hz. The GPS loop sleeps 50 ms after work, so high configured rates are limits on publication attempts, not promises of equivalent fresh measurements. Loss of fix clears the validity indicator; warning messages are rate-limited, and repeated SPI errors trigger recovery attempts. Synthetic position/time and simulator emissions are build-specific test paths.

5.5 Managed-variable synchronization

Both underglow and flight state restore local cached values, register network-variable update callbacks, and then request authoritative values without blocking. Refresh is attempted every 500 ms after network ingress has been seen, until a network value arrives. The flag named `g_telemetry_discovery_seen` is set by ingress callbacks; it is not proof that the complete topology has converged.

Underglow refresh runs from discovery polling; flight-state refresh runs from time-sync polling. Restored values are not injected as newly authoritative network values. Network updates supersede local cache contents.

6 Network inputs, outputs, and contracts

6.1 Router contract

The node identifier is RF. The router has can and radio packed sides, with frame limits of 128 and 1024 bytes respectively and four side-transport templates. SEDSNet owns discovery, route learning, subscriptions, compact framing, and forwarding; application code does not install manual routes or fan out GPS packets.

Both sides are registered without SEDSNet hop acknowledgements, and the router is created with `SEDS_ROUTER_E2E_DISABLED`. Schema reliability flags express message-level protocol metadata; they must not be read as a blanket RFBoard delivery guarantee. CAN has controller retransmission, and the RFD900x link supplies radio acknowledgement/retry. Returned application state remains distinct from either kind of transport acknowledgement.

6.2 What arrives and what RFBoard sends back

Input	RFBoard action	Output/result
Ground commands for valve, actuator, or flight controller	Route toward learned destination on CAN	Destination's response is relayed back; no RF-local execution acknowledgement
Avionics telemetry, warnings, errors, state messages	Route by endpoint subscriptions	Ground-bound traffic exits radio; other destinations depend on discovered topology
AV_BAY_UNDERGLOW	Require one-byte payload; interpret nonzero as on; update PB15 and persist normalized 0/1	Managed-variable handler returns success/error to SEDSNet; local diagnostics update

Input	RFBoard action	Output/result
FLIGHT_STATE	Require one byte in 0–15; update cache and persist changed value	Managed-variable handler success/error; no local actuator action
Managed-variable requests/updates, discovery and time-sync traffic	Dependency-owned protocol handling	Protocol replies, topology advertisements, refresh requests, clock traffic
OTA P2P stream data on port 4510	Validate opcode/state/offset and call LaunchCore	Explicit 13-byte response per received data event; successful finish schedules reset
GPS sentences over SPI	Local acquisition, not network ingress	GPS_DATA, GPS_SATELLITE_NUMBER, configuration message and diagnostic warnings

No board-local network setter exists for telemetry frequency. `RESERVED_135` and `RESERVED_136` preserve legacy schema slots and have no endpoints. `FLIGHT_BUZZER` and `DAQ` calibration are remote-owner schema entries, not additional RFBoard persistent variables.

The complete network schema catalog in Appendix A records payload shapes, endpoints, class, reliability and priority for every type. Numeric data-type IDs and the SEDSNet packed envelope are generated by the pinned dependency; this document deliberately uses symbolic names rather than inventing IDs. Clients must use matching generated bindings/schema.

6.3 UART framing and backpressure

The outer radio frame is:

	Byte offset	Field
	0–1	Sync: A5\5A data, A6\5B command, or A7\7A ASCII-marked frame
	2–3	Payload length, unsigned 16-bit little-endian
	4 onward	1–1024 bytes of payload

The production SEDSNet send path uses the data sync marker. Marker recognition does not create a command shell: received payloads are passed to the router. The parser scans for sync, retains partial frames, rejects zero/oversized lengths, and resynchronizes after invalid data. This outer header has no additional CRC field; integrity behavior within SEDSNet is a separate layer.

RX uses 256-byte chunks and a four-entry ring, with an eight-chunk service budget. The active SEDSNet TX path preserves strict FIFO order with at most 12 entries and 2056 bytes of queued frame storage. When full it reports busy/backpressure; it does not deliberately coalesce or evict compact frames, because later frames may depend on earlier templates. Legacy generic send helpers have different stale/priority handling and are not the router’s priority-send path.

On hardware, TX uses DMA while retaining an owned in-flight frame. Timeout recovery aborts transmit and attempts to requeue the full frame at the front. The normal service starts at most one frame per pass. Firmware simulation substitutes a blocking HAL transmit path, so it cannot fully demonstrate physical DMA behavior.

6.4 CAN packet transport

CAN-FD physical frames carry up to 64 bytes. Larger board-transport messages use an SD fragment header containing magic, source token, sequence, fragment index/count, flags, and total length. The packed header is nine bytes, leaving 55 bytes of data per 64-byte fragment. RF’s fragment source token is 1. Reassembly separates senders using CAN identifier/source/sequence and expires incomplete traffic after 1000 ms.

The configured RX ring is 32 entries, with two reassembly slots of 256 bytes each. These are CMake overrides, not the larger fallback macros visible in the driver. The SEDSNet side’s 128-byte limit is a separate constraint. Outgoing CAN ID selection uses 0x001 for packets recognized as heartbeat and 0x101 otherwise.

CAN enqueue is bounded by a 5 ms timeout. On failure, telemetry can retain three packets of at most 128 bytes. A full pending queue discards its oldest entry and increments a drop counter. Thus acceptance into that queue is not durable delivery; congestion behavior must be included in system command qualification.

7 Persistent and volatile data

7.1 Flash layout

Addresses below are inclusive; KiB means 1024 bytes. The authoritative layout is [board_config.h](#).

Region	Start	End	Size	Role
Bootloader	0x08000000	0x08003FFF	16 KiB	LaunchCore
Slot A	0x08004000	0x08077FFF	464 KiB	Packaged application; vector starts at 0x08004200
Delta region	0x08078000	0x0807DFFF	24 KiB	Staged patch; not a second full-image slot
Metadata 0	0x0807E000	0x0807E7FF	2 KiB	LaunchCore update/boot metadata
Metadata 1	0x0807E800	0x0807EFFF	2 KiB	Alternate metadata region
Persistent data	0x0807F000	0x0807FFFF	4 KiB	Two-bank key/value storage

Slot A reserves 512 bytes before the vector table, leaving 474,624 bytes from the application vector to the slot end. The storage adapter restricts writes to declared application/update/metadata/persistent regions and verifies programmed bytes by readback.

7.2 Durable application values

Value	Key	Representation	Restore/update behavior
Underglow	0x55474C57	One uint8, stored 0 or 1	Missing/invalid data leaves default off; valid cache drives PB15 before scheduler startup; network values normalize nonzero to 1
Flight-state cache	0x46535445	One uint8, accepted 0–15	Missing data means no valid cached value; valid cache restores last-known state; state names are owned outside this repo
Boot/update metadata	LaunchCore-managed	Dependency-defined metadata	Tracks boot/update lifecycle; application requests boot confirmation
Staged delta bytes	Delta partition	LaunchCore delta format	Written during OTA and processed through the LaunchCore update flow

LaunchCore’s fetched bootloader/src/persist.c uses bank generations, committed-bank markers, record sequences, header/value CRC32 checks, append records, and compaction into the alternate bank. This is

the storage mechanism, not proof of complete power-loss qualification for this board. Record metadata consumes flash space even though the two application values total only two bytes.

Changes are applied to RAM/output before flash success is known. A failed write returns a handler error but may leave live state different from the durable value. Repeated unchanged updates normally avoid writes. In particular, flight state updates the RAM comparison value before persistence; if a write fails, a subsequent identical value can bypass the write. Underglow can have the same issue after a previously persisted value exists. A dirty/retry mechanism is proposed in Section 12.

7.3 Volatile state

GPS position/fix/date/time, GPS receiver RAM configuration, router topology, learned subscriptions, time synchronization state, recent packet IDs, queues, fragment assemblies, OTA stream session/offset, and diagnostic counters are volatile. The board does not persist a telemetry history or general packet log. Debug USB output is also not durable storage. GPS receiver configuration is reapplied after boot.

8 OTA update design

OTA is an application-level SEDSNet P2P stream on port **4510**, not a TCP port. Only one connected stream is accepted at a time. All multi-byte integers in this application protocol are little-endian.

Opcode	Request payload	Preconditions/action
01 BEGIN_DELTA	Opcode + uint32 declared size; 5 bytes total	No active update; LaunchCore validates and begins delta reception
02 CHUNK	Opcode + uint32 offset + 1–112 data bytes	Active update; offset equals expected offset; within declared size; offset 8-byte aligned; non-final data length multiple of 8
03 FINISH	Opcode only	Active update; LaunchCore finishes/validates; on success schedules reset
04 ABORT	Opcode only	Active update; abort through LaunchCore
05 STATUS	Opcode only	Return current response fields without changing update state

Every data event produces a response with this layout:

Byte offset	Field
0	Request opcode OR 0x80
1–4	Status uint32
5–8	Next expected byte offset uint32
9–12	Maximum patch size uint32: 24,576

Statuses are 0\OK, 1\BAD_MESSAGE, 2\BAD_STATE, 3\BAD_OFFSET, 4\NO_SPACE, 5\STORAGE_ERROR, 6\BAD_IMAGE, 7\INTERNAL_ERROR. Exact status classification follows [ota_stream.c](#); for example malformed BEGIN length returns BAD_STATE. Chunk offsets advance only on successful storage writes.

Stream close/reset aborts an active receive session. The application has no durable session-resume protocol or inactivity timeout in this handler. A duplicate already-written CHUNK fails the exact-offset check; a client can use STATUS to learn the next expected offset while the session remains alive.

`ota_stream_poll()` requests boot confirmation after 5 s from OTA-service initialization, retrying until LaunchCore succeeds. It does not wait for valid GPS, healthy routing, or complete self-test. A successful FINISH requests reset after 250 ms; this is a grace interval, not confirmation that the remote peer received the final response.

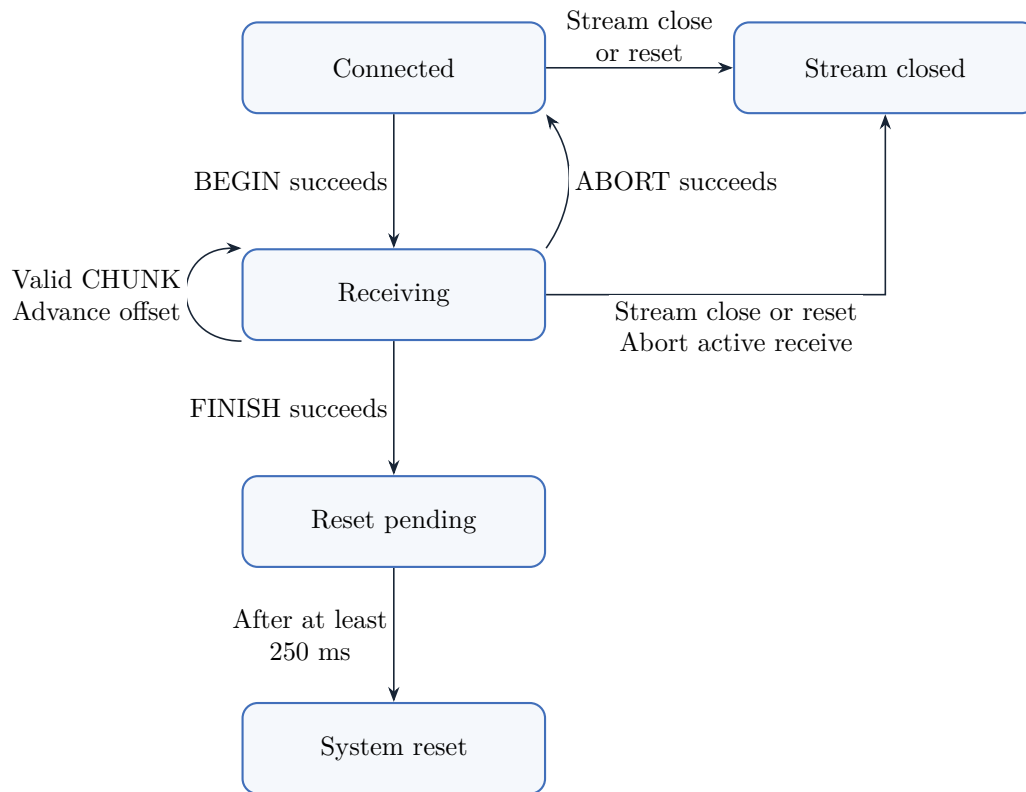


Figure 4: OTA stream lifecycle

Build output uses .seds packages. If a suitable delta cannot be generated or fit, packaging requests full-image recovery. The running application's stream implements delta operations only; full-image recovery must follow the LaunchCore/tooling recovery path, not be sent as arbitrary CHUNK data.

9 Resource and timing budgets

Resource	Configured allocation/limit	Interpretation
MCU SRAM	112 KiB	Board layout capacity; actual linked margin must be measured
Application ThreadX pool	66,264 bytes	Includes normal SEDSNet arenas, telemetry stack, and other allocations
SEDSNet total arenas	67,072 bytes	22,528 small + 25,088 large + 19,456 static emergency
SEDSNet normal arenas	47,616 bytes	Allocated inside application pool; do not add again to RAM total
SEDSNet queue budget	6144 bytes	Runtime bounded queue storage; not the total allocator capacity
Starting queue storage	512 bytes	Initial fallback storage
Recent RX IDs	16	Bounded duplicate/history bookkeeping
Debug USB pool	10,752 bytes	Conditional; USBX memory stack 8192 bytes and app thread stack 2048 bytes
GPS static stack	12,288 bytes	Outside application pool
Telemetry stack	14,336 bytes	Inside application pool
UART TX queue	12 entries / 2056 bytes	Plus separate active-frame and other driver storage

The allocator segregates allocations of at least 1 KiB into the large arena, tries alternate normal storage on failure, then emergency storage. Allocation uses non-waiting ThreadX calls. Arenas, nested pools,

stacks, static buffers, middleware, and linker sections must be accounted without double-counting; this table is not an ELF memory report.

At 57,600 baud with 8N1, the UART has a theoretical ceiling of 5760 bytes/s before radio framing and network overhead. A maximum 1028-byte outer frame takes approximately 178.5 ms on the UART alone. These are calculated serialization bounds, not measured RF throughput. CAN runs much faster, making CAN-to-radio load control a central constraint.

Required engineering measurements include worst-case service-pass duration, command round-trip time under discovery and OTA load, ISR-ring overflow under continuous ingress, stack low-water marks, allocator low-water marks, and behavior at the selected telemetry rate. Current source settings establish budgets but not hard real-time guarantees.

10 Fault handling and trust boundaries

Condition	Implemented handling	Practical limit
GPS no fix / parse / SPI trouble	Clear fix indication, bounded warnings, SPI recovery	Valid position availability is not guaranteed; no automatic synthetic production fallback
UART malformed framing	Reject bad lengths and search for sync	Header framing alone does not authenticate content
UART DMA stall	Timeout, abort, front requeue attempt	Requeue can fail; physical behavior requires hardware testing
CAN congestion	Bounded HAL enqueue and three-packet retry queue	Oldest pending packet can be dropped
CAN incomplete/interleaved fragments	Bounded reassembly slots and 1 s expiry	Excess concurrent traffic can replace an older partial assembly
Router allocation/setup failure	Error result and generally delayed reinitialization	Some callers discard errors; successful scheduler start is insufficient evidence of healthy routing
Persistent record error	Reject invalid restore, increment counters; handler error on write failure	Live value can differ from flash value
Thread stack overflow / fatal HAL failure	Capture diagnostics and enter fatal LED loop	No automatic recovery is established
Debug USB unavailable	Drop output/best-effort behavior	Logs cannot be relied on as a complete event record

SEDSNet cryptography is explicitly disabled in `sedsnetch.fetch.cmake`. Board-local OTA and managed-variable handlers do not perform sender authorization. Reliable delivery, CRC checks, and image-format verification must not be treated as proof of authenticated command/update origin. A mission deployment needs an explicit decision about trusted radio/CAN access and update authorization.

The board boot hook platform `_feed_watchdog()` is empty, and no active application hardware-watchdog setup was identified. A fatal loop may therefore require external intervention. This is an implementation fact to address in system availability requirements, not an assertion that a watchdog is already present.

11 Build, deployment, and verification

11.1 Reproducible configuration

The build uses CMake/Ninja, an ARM toolchain, Python build tooling, and Rust target thumbv7em-none-eabihf. CMake fetches SEDSNet **v4.0.27** and LaunchCore **v1.0.0**. These are baseline pins, not claims about the newest upstream releases. SEDSNet schema generation consumes `config/sedsnetch.json`.

CubeMX's .ioc owns generated RTOS pool settings. The board-owned top-level CMake reconnects generated MCU code to application modules, generated LaunchCore linker scripts, and external libraries. Release disables USB debug and enables optimization/LTO; test/simulator flags must be recorded with qualification results.

```
# Run from RFBoard26, not from the sdd folder.
./build.py build --release
./build.py test
./build.py test --all --release
./build.py test --all --release --ultra-soak
```

The documented default flash command is `./build.py\flash\-\-release\-\-method\dfu`; it writes the complete factory image at 0x08000000. Application-only flashing assumes an already compatible bootloader and metadata. Do not treat ordinary flash replacement as proof that application-persistent values survive every programming method.

11.2 Design-to-test traceability

Objective / design contract	Existing verification source	What still needs system/hardware evidence
D-01 two-sided routing, automatic routes	Routing contract , qualification contract , linked simulator	Physical radio/CAN loss, reconnect, command response latency
D-02 telemetry rate	Rate tests , GPS qualification checks	Fresh-sample rate, GPS reception and antenna behavior
D-03 time/discovery startup	Qualification tests and simulator network probes	UTC accuracy, source loss/reacquisition, long uptime
D-04 persistence	Flight-state tests , handoff tests , LaunchCore persistence tests	Power-cut sweep, wear, failed-write retry behavior
D-05 update layout/protocol	OTA contract , delta format tests	Interrupted update/install, recovery, authorization, final-response delivery
D-06 memory/congestion	Memory contracts , CAN contract , simulator memory/stack probes	DMA timing, flash stall impact, peak stack usage on target
D-07 build and deployment	Build tests , runner tests , CubeMX checks	Repeatability of production flashing and hardware qualification

The inspected working-tree schema and network-schema contract test both specify Ordered delivery for UMBILICAL_STATUS. This differs from the versioned baseline, which used Unordered; see Appendix A.

Many Python tests inspect source/configuration contracts; they are not behavioral proof of the entire firmware. The linked simulator uses the actual ELF and checks discovery, traffic attribution, command/state ACK correlation, and memory. The documented normal linked test is 16 s; ultra-soak adds 600,000 ms of firmware-time fault/rejoin and command/ACK testing.

Simulator flags introduce deterministic time/probes and satellite publication. The explicit `sim_probe_emit_heartbeat()` helper emits at most four test heartbeats and is gated by `SEDS_FIRMWARE_SIM_TEST`; it is not evidence of an application-owned periodic production heartbeat task.

For this documentation task, source and configuration were inspected and document links/catalog consistency were checked. The expanded LaTeX edition was also compiled locally; this is a document build, not firmware qualification. Firmware was not rebuilt, flashed, or requalified. The commands above describe the repository's verification workflow, not test runs performed while writing this document.

12 Open engineering work

These are proposed follow-up tasks, not changes made by this document.

Priority	Task	Why / acceptance evidence
High	Define a command delivery/ACK contract under queue pressure	Account for CAN oldest-entry loss and disabled RF router E2E; demonstrate execution/response correlation, timeout and retry behavior under sustained load

Priority	Task	Why / acceptance evidence
High	Make failed persistent writes retryable	Retain dirty state until committed; inject write failures and repeat identical values; verify reset restores the last successfully committed value
High	Define update/command authorization	Document trusted peers and transport assumptions; demonstrate rejection of unauthorized updates if required by the deployment
High	Tie boot confirmation to agreed health criteria	Decide whether router initialization, loop progress, storage health and other checks must precede confirmation; test a boot that schedules but cannot route
High	Qualify power loss and full recovery	Interrupt receive, metadata writes, compaction and installation; verify supported recovery path without assuming a full Slot B image
Medium	Check and surface flight-state subscription initialization errors	<code>flight_state_cache_init(r)</code> return is currently ignored; demonstrate retry/diagnosis instead of silently missing updates
Medium	Measure flash blocking and scheduling margins	Run managed-variable writes and OTA while routing commands; establish measured worst-case latency and queue occupancy
Medium	Define watchdog/recovery policy	Decide whether fatal loops should reset and what health progress gates watchdog feeding
Medium	Define time validity and long-uptime behavior	Exercise source loss, UTC discontinuity and 32-bit tick wrap; document timestamp validity consumers can rely on
Medium	Complete interface semantics with peer owners	Specify flight-state 0–15 names, command codes, units absent from schema, sender authority, and response correlation
Medium	Qualify Debug RAM and production observability	Verify linked margins with USB enabled; determine which diagnostics should be accessible in Release
Low	Keep driver/task documentation current	Recheck CMake overrides and generated settings whenever stacks, queues, schema, middleware, or CubeMX output changes

13 Design rationale and references

Decision	Rationale visible in implementation
Separate CAN/radio sides with learned routing	Preserve ingress identity and let discovery/subscriptions determine forwarding
Avoid nested radio hop ACK queues	RFD900x already retries; extra queues previously retained stale control traffic and starved application delivery
Preserve compact-frame FIFO order	Template/discovery dependencies can make reordering or coalescing corrupt subsequent decoding
Separate GPS and network threads	Blocking acquisition can yield without making the GPS loop the network service owner
Restore cache before networking	Preserve local output/state through reset while awaiting authoritative synchronization
Fixed allocator/queue budgets	Contain RAM consumption on a 112 KiB device and expose pressure through instrumentation
Compile-time publication rate	Keep board cadence explicit in the build and avoid obsolete network-rate variables

The document’s organization was informed by NASA’s guidance on component responsibilities, execution,

interfaces, resource use, and traceability. This is structural guidance, not a claim of NASA process compliance. [NASA Software Design Description guidance](#).

The boot, command round-trip, and update scenarios use the runtime-view approach described by arc42. Its HTML Sanity Checker example illustrates documenting a scenario through participating components and their sequence of actions; this document applies that presentation approach to embedded firmware. [arc42 runtime-view guidance](#), [HTML Sanity Checker runtime design example](#).

Additional structural reference: arc42's component/building-block view, used here to connect module responsibilities to source files. [arc42 building-block view](#).

The external references informed presentation only. All RFBoard-specific behavior and numeric settings come from the inspected repository baseline. Locally fetched LaunchCore implementation was inspected under build/Release/_deps/sedslaunchcore-src; that generated path is evidence for the pinned dependency, not a required portable documentation link.

Glossary: ACK — acknowledgement; CAN-FD — CAN with flexible data rate; CDC ACM — USB virtual serial class; DMA — direct memory access; HAL — hardware abstraction layer; NMEA — GPS receiver sentence format; OTA — over-the-air firmware update; P2P — peer-to-peer; SDD — software design document; UTC — coordinated universal time.

14 Detailed execution and driver design

This section expands the implementation baseline into reviewable algorithms. Pseudocode describes ownership and ordering, not a replacement implementation. Steps marked **proposed** are engineering changes that have not been made to the firmware.

14.1 Interrupt-to-task handoff

Interrupt priorities and task priorities belong to different schedulers and must not be numerically compared. The generated NVIC settings assign USB priority 5, CAN interrupt lines priority 6, and USART1/DMA interrupt lines priority 7. ThreadX schedules the telemetry thread ahead of the GPS thread after interrupt handling returns. The actual interrupt latency also depends on interrupt masking, HAL execution, and higher-priority activity.

The interrupt layer should complete the hardware event and preserve received bytes or completion state. The telemetry task performs parsing, reassembly, router calls, and egress decisions. Keeping those actions in thread context permits mutex use and limits the amount of application processing performed inside an interrupt. A bounded ring only absorbs a finite service delay; it does not create unlimited receive capacity.

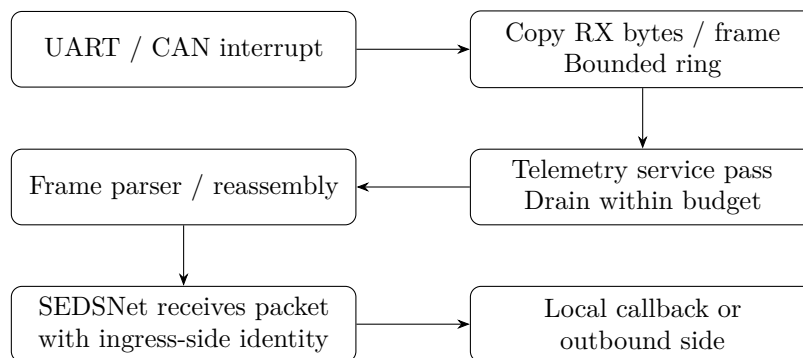


Figure 5: Interrupt-to-task receive handoff

Evidence: interrupt handlers in Core/Src/stm32g4xx_it.c, callbacks in radio.c and can_bus.c, and the service order in telemetry_thread.c. An overflowing receive ring means bytes or frames can be lost before SEDSNet sees them. The next layer must tolerate invalid/incomplete input, and system tests must distinguish this from an application handler rejecting an otherwise valid packet.

14.2 Radio receive algorithm

The UART is a byte stream: DMA idle events do not necessarily align with packet boundaries. One callback may contain part of a frame, several frames, or a suffix of one frame followed by another frame. The parser therefore maintains a persistent assembly buffer across receive calls.

The parser's essential sequence is:

1. Add newly received bytes to bounded frame storage.
2. Search for one of the recognized two-byte synchronization markers.
3. Discard preceding non-frame bytes. Preserve a trailing possible first sync byte when the second byte has not arrived.
4. Wait until the four-byte outer header is available.
5. Decode the little-endian length; reject zero or values above 1024.
6. Wait until the full payload is present.
7. Notify subscribers with the payload and current frame classification.
8. Consume that frame and continue examining any remaining bytes.

A parser rejection should not reset the whole network router. Conversely, successful outer framing does not mean that the payload is a valid SEDSNet packet. The transport and router have separate validation responsibilities. The frame's command marker is classification information; it does not grant authorization or bypass router parsing.

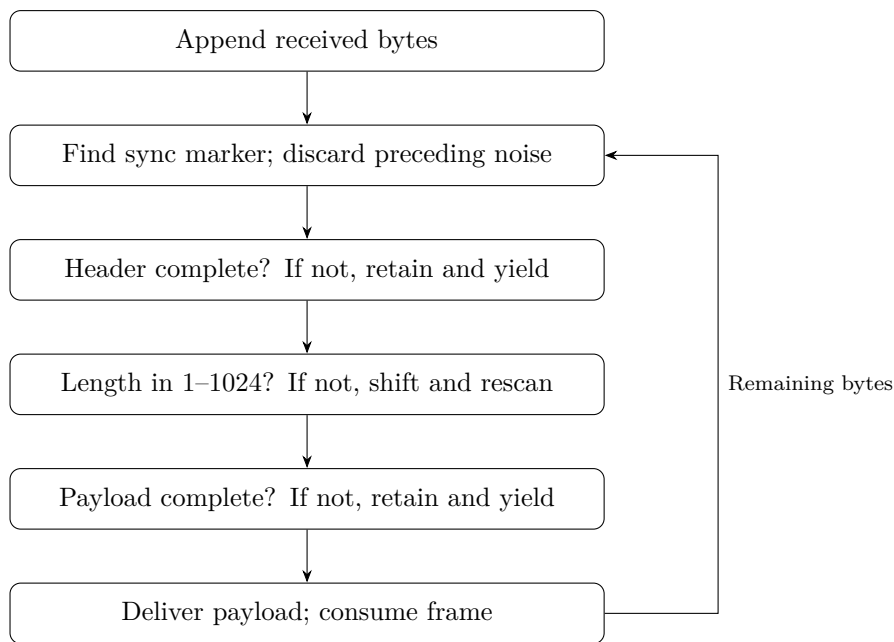


Figure 6: UART frame parsing and resynchronization

Recommended receive tests split each header at every byte boundary, split payloads across multiple DMA events, concatenate frames, insert garbage between frames, and inject invalid length fields. Each test should verify both recovery and the absence of accidental double delivery.

14.3 Radio transmit lifecycle

The router's priority callback calls `radio_uart_send_bytes_priority()`. That function builds the four-byte outer header and copies the complete frame into the strict SEDSNet FIFO. A successful call means ownership was accepted by the driver. A busy result tells the upper layer to retain responsibility and retry according to its protocol behavior.

The queue and in-flight DMA item are different storage objects. Dequeue transfers bytes into the in-flight item; DMA reads from that retained storage until completion. Releasing or reusing it earlier

would corrupt the wire frame. The completion callback clears busy state. The telemetry service examines elapsed time to detect a missing completion and attempts recovery from the full frame's beginning.

A complete retransmission after an uncertain UART completion may repeat bytes that were partly transmitted before the abort. The receiver must resynchronize, and higher layers must tolerate possible duplication. This is another reason that transport completion cannot be substituted for a destination's executed-command response.

The legacy queue helper contains stale-age and coalescing rules. Those rules are not the current compact-frame FIFO contract. Reviews must trace the active caller, not infer production behavior from an unused helper or a macro name alone.

14.4 CAN fragmentation and reassembly

Fragmentation converts a transport packet larger than one physical CAN frame into several independently arbitrated frames. The nine-byte fragment header reduces usable fragment payload to 55 bytes. For a 128-byte transport packet, the fragmentation path requires three frames: two containing 55 data bytes and one containing the remaining 18 data bytes, with physical padding governed by the driver.

A reassembly slot records the CAN ID, source token, sequence, fragment count, total length, last-received tick, received-fragment bitmap, and reconstructed bytes. Matching uses sender identity and sequence so that interleaved traffic does not append one board's bytes to another board's packet. When a sender's sequence changes, the old partial assembly is reset. When no slot is free, the oldest assembly can be replaced. Incomplete assemblies expire after 1 s.

The advertised router-side frame limit, physical CAN payload size, and reassembly capacity are intentionally different numbers. Enlarging one without updating the others can create traffic that one layer accepts but the next layer cannot carry. Any capacity change requires checking CMake definitions, actual static RAM, fragment bounds, simulator traffic, and peer compatibility together.

14.5 GPS parsing, freshness, and receiver recovery

`receive_nmea()` asserts chip select, searches for `\protect\TU\textdollar`, reads the five-character talker/type header and separator, and dispatches supported GGA or RMC sentence handling. GPS position conversion changes degrees/minutes text into signed decimal degrees. Hemisphere fields determine the sign. GGA supplies fix quality, satellite count and mean-sea-level altitude. RMC supplies status, position, date and time, and leaves previously acquired altitude intact.

The result is a stateful sample assembled over multiple sentence types. A newly received RMC position can coexist with an older GGA altitude. The current packet type has no per-component age field. Consumers should not assume all three position components were measured at exactly the same instant. A proposed stronger sample contract would track timestamps for GGA and RMC separately and publish an explicit age/validity indication.

A fresh satellite field records a HAL tick. A satellite field older than 5 s, or a field that has never been updated, produces a zero satellite count. A newly observed satellite update can force publication before its ordinary due time; consequently the configured rate is not a strict minimum interval for every satellite event. Position and satellite publication maintain separate deadlines, and successful publication advances the relevant deadline.

The thread's error controls are specific:

Condition	Threshold / interval	Result
Initial GPS link has not been established	5 min	Initial-link warning once through that path
Previously established fix is absent	100 no-fix iterations	Warning once per lost-fix episode
Repeated receive errors	20 consecutive errors	Rate-limited error reporting by error category
Repeated SPI errors	At least 3 consecutive errors; recovery attempts separated by 5 s	Abort SPI, deinitialize, briefly yield, reinitialize

Condition	Threshold / interval	Result
Repeated error-category warning	5 min per category	Avoid filling the radio with repeated diagnostics

Iteration thresholds must not be converted directly into wall-clock guarantees: acquisition and scheduling take variable time. The source’s NMEA field parsing also deserves malformed-input/checksum tests; successful field extraction alone does not establish end-to-end sentence integrity.

15 Detailed data consistency and state design

15.1 Lifecycle of a managed value

A managed value has several distinguishable states: no valid local cache, a restored cache awaiting synchronization, a live network value, and a live value whose flash write failed. The current application tracks some of those conditions through booleans and counters rather than a single explicit state machine.

The following diagram separates implemented transitions from a proposed persistence-retry path. The last transition is an improvement to implement, not a claim that current firmware retries automatically.

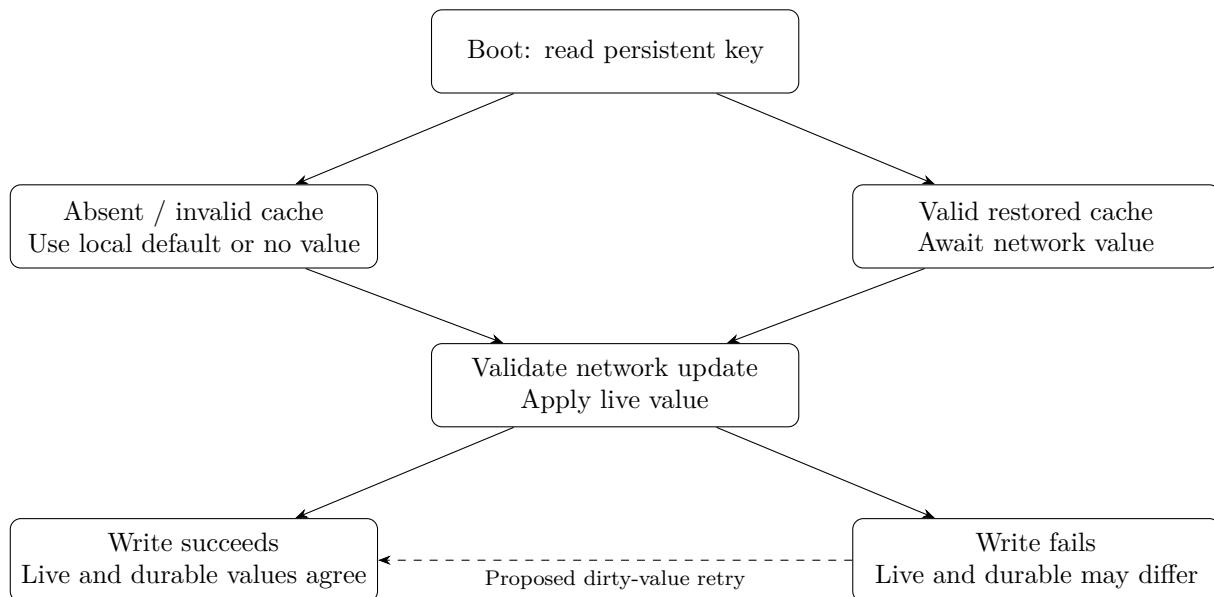


Figure 7: Managed-value lifecycle and proposed persistence retry

For underglow, applying live state changes a physical output. For flight state, it updates a cache and invokes the weak hook. The two types therefore have different externally observable effects despite sharing the same persistence adapter. This distinction matters in failure tests: an underglow error should record both the observed GPIO and the value restored after reset, while a flight-state test should record cache validity and value.

15.2 Two-bank persistence algorithm

The persistent partition consists of two 2 KiB banks. On initialization, LaunchCore reads each bank header and accepts only headers satisfying its version, size, committed-marker and CRC checks. If both are valid, the higher generation is selected. If neither is valid, the implementation formats an initial bank. Application-level restoration then looks up each key and validates its value size/range.

Ordinary writes append aligned records containing key, sequence, flags, value size and CRC metadata. When space is insufficient, compaction prepares the alternate bank with a newer generation, scans the source, copies each latest non-deleted record, and commits the target bank. The source is not simply erased before its values are copied.

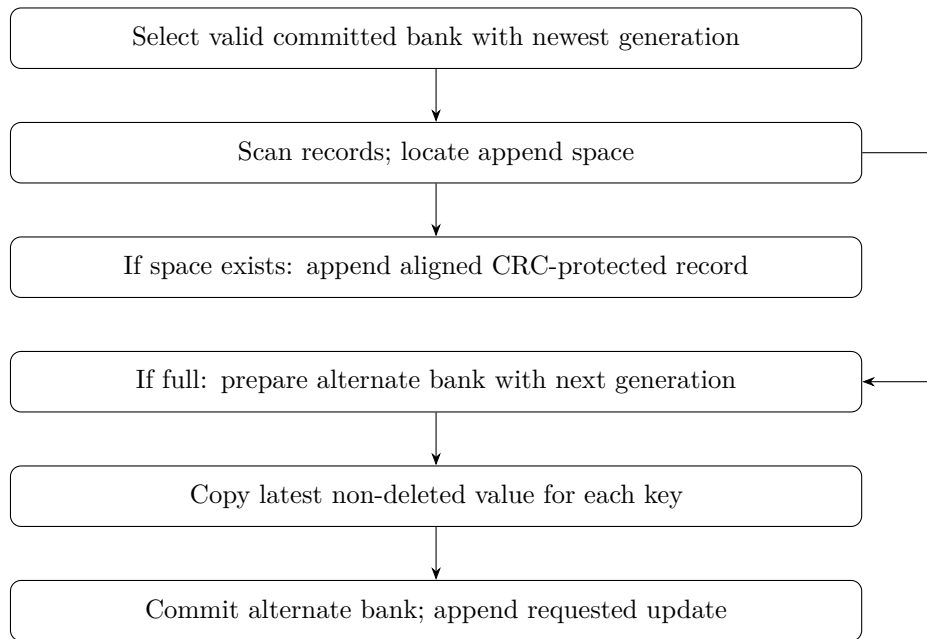


Figure 8: Two-bank persistent-store append and compaction flow

The algorithm provides explicit commit boundaries that can be tested. It does not prove that every interrupted operation leaves a readable latest value. Flash write granularity, partial programming, corrupted records, generation selection, and the adapter’s readback checks must all be exercised with injected failures or actual power interruption. In particular, a test should distinguish retaining the previous committed value from retaining the most recently attempted value.

15.3 Proposed persistence retry contract

A robust follow-up design would keep `live_value`, `committed_value`, and a dirty flag separately. Apply a validated network value to live state; set dirty if live differs from committed; attempt a bounded write; update committed state and clear dirty only after success. Identical repeated network updates must not clear a dirty value. A retry timer should avoid continuous flash retries on a permanent storage fault.

The retry design also needs a defined response policy. An acknowledgement might mean either “applied in RAM” or “durably committed”; those meanings must not be mixed. If network-variable protocol acknowledgements cannot carry durability semantics, publish a separate storage-health observation or document that the acknowledgement covers live application only. This is a proposed system contract requiring agreement with the ground software.

15.4 Clock representation and consistency

Three time representations appear in the board: raw local ticks, Unix epoch milliseconds, and GPS time-of-day offset. Monotonic service timing should use local elapsed time. Telemetry consumers need an epoch and a validity indication. A time-of-day offset cannot establish the date by itself.

The legacy offset helper currently rejects measured offsets outside plus/minus 60,000 ms and adds measured_offset/8 to its existing value, with a minimum signed one-millisecond step for a nonzero remainder. It is not a conventional filter that adds a fraction of (measured - current). This distinction should be documented before any future consumer treats the helper as an accurate continuously converging clock. Production network epoch updates use `telemetry_set_unix_time_ms()` directly and should be evaluated separately.

The helper exposes a volatile 64-bit offset on a 32-bit MCU. Volatile access does not itself promise atomic snapshots. Any new cross-task consumer should introduce an atomic/synchronized snapshot mechanism rather than assume that two 32-bit accesses cannot be interrupted. Likewise, widening a 32-bit tick to

64 bits does not extend its wrap period. At 1000 ticks/s, a 32-bit tick wraps after approximately 49.7 days; long-uptime tests should include that boundary.

16 Interface contracts and integration examples

16.1 API responsibilities

Interface	Caller responsibility	Callee responsibility / result
<code>log_telemetry_asynchronous()</code>	Supply correct schema type, element count and element size; use thread context	Submit through SEDSNet; return status; success is not remote execution
<code>radio_uart_send_bytes_priority()</code>	Supply complete side-transport bytes within frame limit	Copy framed bytes into bounded FIFO or return busy/error
<code>can_bus_send_large()</code>	Supply a compatible payload and CAN identifier	Send/fragment within hardware limits; return HAL status
<code>persistent_store_get()</code>	Supply key, output buffer and size capacity	Return stored bytes/status; caller validates application range
<code>persistent_store_set()</code>	Supply valid key/value; accept possible synchronous flash delay	Serialize application persistence access and return storage status
<code>flight_state_cache_init()</code>	Supply live router; check result	Enable variable and register callback; failure must not be ignored in a stronger design
<code>ota_stream_init()</code>	Supply live router after sides exist	Initialize storage context and bind application stream port
<code>ota_stream_poll()</code>	Call repeatedly from a live telemetry loop	Attempt delayed boot confirmation and deferred reset

These responsibilities do not change dependency ownership rules. A caller must not keep a borrowed callback payload after its owner releases it, publish from an interrupt that cannot block, or interpret an enqueue status as a physical transfer receipt.

16.2 Example: valid underglow update

Assume a one-byte underglow value of 1 arrives through a learned radio route. SEDSNet recognizes the managed-variable update and invokes the local callback. The callback verifies the type, presence and one-byte length, normalizes the byte to on, and drives PB15 high. It marks that a network value has been seen, so startup refresh requests stop. If persistence is needed, it writes a single normalized byte under key 0x55474C57.

If flash succeeds, the handler returns success and both the live output and durable setting agree. If flash fails, the output can still remain on while the callback returns an error and the persistent error counter increments. After reset the previously committed value may be restored. Testing only the network return code would miss the output-side effect; testing only the GPIO would miss the persistence failure.

16.3 Example: OTA chunk progression

Consider an illustrative 224-byte patch accepted by BEGIN. The request is 01\E0\00\00\00. After a successful BEGIN, the response contains opcode 81, status 0, expected offset 0, and maximum size 24,576. The maximum-size field is serialized as 00\60\00\00.

The client sends CHUNK opcode 02, offset 00\00\00\00, and 112 patch bytes. After success, the expected offset is 112 (70\00\00\00). The next chunk begins at 112 and supplies the remaining 112 bytes. FINISH then asks LaunchCore to validate completion and image content. These byte examples explain the application envelope; they do not constitute a valid delta image by themselves.

If the first chunk's response is lost, blindly repeating offset zero can return BAD_OFFSET because the server already advanced to 112. STATUS exposes the expected offset within the current session.

If the stream is closed or reset, the application aborts reception rather than advertising durable reconnect/resume semantics. A client must distinguish retrying a request on the same connection from restarting an aborted update.

16.4 Discovery, reconnect, and stale traffic

Routes and side-transport templates are dynamic RAM state. After a reset or reconnect, a peer may need discovery and template exchange before ordinary compact application traffic can be decoded or routed. Forcing a large topology snapshot every service pass is not the implemented design; SEDSNet controls its adaptive discovery cadence.

The board registers both sides before time/discovery services so the initial network view can include the complete relay. GroundStation preference concerns discovery organization, while GPS time-source priority concerns clock selection. A test that sees one incoming packet or one valid clock sample has not necessarily demonstrated topology convergence, successful command forwarding, and returned application state.

17 Qualification scenarios and acceptance planning

The following scenarios expand the existing contract tests into a verification plan. They are proposed acceptance work, not a report that the tests have already passed. Numerical mission-level latency and recovery limits remain to be assigned by the team.

17.1 Startup and synchronization scenarios

Scenario	Setup / stimulus	Observe	Acceptance intent
Cold boot with empty persistent storage	Erased persistent region; normal peers	GPIO default, cache validity, service-loop progress, discovered sides	No invalid cached state; normal routing becomes available
Warm boot with known cache	Store underglow on and a valid flight state; reset	Output before scheduler; later network update	Cached state restored before synchronization; authoritative value replaces it
Boot without radio peer	CAN active, no GroundStation connection	Queue pressure, GPS progress, memory low-water	Service stays responsive and does not grow unbounded queues
Late GroundStation connection	Bring radio peer online after startup	Discovery, templates, bidirectional payload	Converges without reset or manual route installation
GPS absent at boot	Missing or unresponsive receiver	Router progress and bounded warnings	Relay remains serviceable while GPS is unavailable
Time source disappears	Stop valid source updates after synchronization	Clock validity and source selection	Documented behavior during loss and recovery; no invented timestamp validity

17.2 Parser and transport scenarios

Scenario	Injection	Required observation
UART split/merged frames	Every header split, multi-frame chunks, partial payload	Correct packet count and bytes; no duplicate callback
UART garbage and bad length	Noise, zero length, oversized length	Parser resynchronizes; router survives
CAN interleaving	Multiple senders and sequences	No cross-sender assembly corruption
CAN missing fragments	Drop one fragment, wait past expiry	Partial packet is not delivered as complete
Reassembly exhaustion	More concurrent assemblies than slots	Bounded memory and documented replacement behavior
UART completion failure	Suppress TX completion in target-aware test	Timeout recovery or reported drop; later traffic can progress
CAN queue saturation	Block bus acknowledgements/transmit resources	Bounded retry behavior and observable drop count

Scenario	Injection	Required observation
Compact-frame dependency	Discovery/template followed by dependent payload	FIFO preserves decoding dependencies through backpressure

17.3 State, flash, and update scenarios

For each persistent key, test missing data, wrong length, out-of-range stored bytes, repeated identical updates, changed updates, and failure during the first and subsequent writes. Record the handler result, live state, diagnostic counters, and post-reset restored state. Those four observations describe different parts of the contract.

For storage compaction, fill enough records to cross a bank boundary and interrupt execution before target preparation, during record copy, before commit, and after commit. Confirm which generation and key values are selected on reboot. Include invalid CRC and truncated-record cases. A host storage mock can identify algorithmic defects; a physical power-cut test covers adapter/flash behavior that a mock cannot reproduce.

For OTA, vary declared size, opcode, stream identity, offset, alignment, chunk length and session lifetime. Exercise chunk sizes 1, 7, 8, 111, 112 and 113 in both final and non-final positions. Verify that invalid non-final alignment does not advance the offset, that 113 bytes is rejected, and that final unaligned lengths are handled according to the LaunchCore adapter contract. Include an otherwise well-framed but invalid image and verify `BAD_IMAGE` when the dependency reports verification failure.

17.4 Performance and soak evidence

A useful run artifact contains the source commit, dependency pins, compiler/build type, simulator flags, telemetry rate, queue/pool configuration, topology, test duration, fault schedule and raw results. Report worst observed service-loop gap and command round-trip time rather than only average throughput. Track allocator low-water, peak stack use, queue occupancy, drops, parser errors, DMA recoveries, and persistent errors over time.

An allocator snapshot that remains below capacity is not alone a leak test. Compare steady-state memory before load, during load and after reconnection/drain, and observe whether repeated fault cycles consume progressively more memory. Similarly, a changing scheduler tick is not evidence that the full routing loop is making progress; the loop-completion counter exists to distinguish those conditions.

Physical qualification should run production Release firmware with the actual UART DMA path, radio hardware and CAN transceivers. Simulator results remain valuable for repeatable topology, command correlation, fault injection, and memory observations, but the simulator's alternate UART transmit path means target timing evidence is still required.

18 Maintenance and implementation guidance

18.1 Change-impact checklist

Changes to the shared schema affect generated bindings, peer compatibility, routing subscriptions, ground decoding and tests. Preserve reserved slots unless the schema migration explicitly coordinates all peers. If payload element counts or types change, revise both the catalog and the publishing/consuming code. Do not treat a message's presence in the schema as proof that RFBoard handles it locally.

Changes to queues or allocator arenas affect static RAM and nesting within the application byte pool. Regenerate and inspect the ELF memory report rather than sum every configured pool as if each were independent. Changes to thread stacks must account for deeper dependency calls, interrupt/preemption margin and Debug versus Release differences.

Changes to flash layout affect the bootloader, application vector address, generated linker scripts, OTA package tooling, delta capacity, metadata and persistence. Keep the source of truth in the board layout

and verify alignment/non-overlap tests. An application built for a new vector/slot layout must not be deployed onto an incompatible installed bootloader.

Changes to control flow should preserve CAN initialization before router setup, both side registrations before network services, and opportunities to process both ingress and egress. Any new blocking operation in a callback should be reviewed against the radio backlog budget. If a new storage worker is introduced, define ownership and completion semantics explicitly; no such worker exists in the present baseline.

18.2 Recommended implementation sequence

First agree on externally visible contracts: command acknowledgements, variable durability, time validity, update authorization and boot health. Then implement targeted improvements with behavioral tests for failed persistence, subscription initialization failure, and queue-pressure command handling. Next qualify flash/update interruption and target DMA recovery. Finally establish measured timing budgets and maintain them in the build/qualification record.

The design does not require creating more tasks simply to match the number of modules. A new task is justified when it has an independent scheduling requirement or must isolate bounded work from the router service. Every additional task consumes stack and introduces synchronization, lifetime and shutdown/restart decisions. On this RAM-constrained MCU, task boundaries should follow timing and ownership needs.

A Network schema catalog

Source commit baseline: b4eba479b57f767d07c7c9a811c741bc043990e6, inspected 2026-09-11. Generated from [config/sedsnet.json](#). See the main document for actual RFBoard ownership, routing, framing and persistence.

This catalog includes shared network types. A listed sensor or endpoint does not imply local hardware, publication, storage, or command execution on RFBoard. Reliability and priority below are schema declarations; actual delivery also depends on router configuration and link/queue behavior. Numeric wire IDs are generated by SEDSNet and are not assigned here.

A.0.0.1 Working-tree schema note. The catalog includes the inspected working-tree setting for UMBILICAL_STATUS: reliable Ordered delivery. The baseline commit used Unordered. The inspected working-tree schema and its network-schema contract test now both specify Ordered. This document records that working-tree change without modifying firmware or claiming a qualification run.

A.1 Endpoints

Name	Description	Broadcast mode
SD_CARD	On-board storage (e.g. SD card / flash).	Schema default
GROUND_STATION	Data destined for the groundstation	Schema default
FLIGHT_CONTROLLER	Data destined for the Flight Computer	Schema default
VALVE_BOARD	Data destined for the Valve Board	Schema default
ABORT	Endpoint For abort commands	Schema default
FLIGHT_STATE	Flight state updates	Schema default
HEART_BEAT	Endpoint for heartbeat messages	Default
ACTUATOR_BOARD	Actuator board endpoint	Schema default
AV_BAY_UNDERGLOW_OWNER	Boards that own the persistent avionics-bay underglow variable	Schema default

A.2 Data types

Static payload sizes are decoded element bytes, excluding packet headers, compression and transport framing. Dynamic string lengths are variable. Class, reliability and priority are grouped so this catalog remains readable on portrait pages.

Type	Payload / size	Delivery	Destinations
GENERIC_ERROR	Dynamic String Variable bytes	Error true / Ordered Priority 0	SD_CARD, GROUND_STATION
GPS_DATA	3 × Float32 12 bytes	Data false / None Priority 0	GROUND_STATION, SD_CARD, FLIGHT_CONTROLLER
GYRO_DATA	3 × Float32 12 bytes	Data false / None Priority 0	GROUND_STATION
ACCEL_DATA	3 × Float32 12 bytes	Data false / None Priority 0	GROUND_STATION
BATTERY_VOLTAGE	1 × Float32 4 bytes	Data false / None Priority 0	GROUND_STATION, SD_CARD
BATTERY_CURRENT	1 × Float32 4 bytes	Data false / None Priority 0	GROUND_STATION, SD_CARD
BAROMETER_DATA	3 × Float32 12 bytes	Data false / None Priority 0	GROUND_STATION
ABORT	Dynamic String Variable bytes	Error true / Unordered Priority 0	ABORT, SD_CARD
FUEL_FLOW	1 × Float32 4 bytes	Data false / None Priority 0	GROUND_STATION, SD_CARD
VALVE_COMMAND	1 × UInt8 1 bytes	Data true / Ordered Priority 0	GROUND_STATION, VALVE_BOARD
FLIGHT_COMMAND	1 × UInt8 1 bytes	Data true / Ordered Priority 0	GROUND_STATION, SD_CARD, FLIGHT_CONTROLLER
FUEL_TANK_PRESSURE	1 × Float32 4 bytes	Data false / None Priority 0	GROUND_STATION, SD_CARD
FLIGHT_STATE	1 × UInt8 1 bytes	Data true / Ordered Priority 16	FLIGHT_STATE, SD_CARD, GROUND_STATION
HEARTBEAT	0 × NoData 0 bytes	Data false / None Priority 255	HEART_BEAT, GROUND_STATION, FLIGHT_CONTROLLER
WARNING	Dynamic String Variable bytes	Warning true / Ordered Priority 0	SD_CARD, GROUND_STATION
MESSAGE_DATA	Dynamic String Variable bytes	Data false / None Priority 0	GROUND_STATION
ACTUATOR_COMMAND	1 × UInt8 1 bytes	Data true / Ordered Priority 0	ACTUATOR_BOARD, GROUND_STATION
UMBILICAL_STATUS	2 × UInt8 2 bytes	Data true / Ordered Priority 200	GROUND_STATION
KG1000	1 × Float32 4 bytes	Data false / None Priority 0	GROUND_STATION

Type	Payload / size	Delivery	Destinations
KG50	1 × Float32 4 bytes	Data false / None Priority 0	GROUND_STATION
GPS_SATELLITE_NUMB ER	1 × UInt8 1 bytes	Data false / None Priority 0	GROUND_STATION
EULER_ANGLES	3 × Float32 12 bytes	Data true / Unordered Priority 0	SD_CARD, GROUND_STATION
ORDERED_MESSAGE	Dynamic String Variable bytes	Data true / Ordered Priority 0	SD_CARD, GROUND_STATION
ASCENT_STATE	6 × Float32 24 bytes	Data false / None Priority 0	GROUND_STATION
DESCENT_STATE	4 × Float32 16 bytes	Data false / None Priority 0	GROUND_STATION
ASCENT_BIASES	4 × Float32 16 bytes	Data true / Unordered Priority 0	SD_CARD, GROUND_STATION, FLIGHT_CONTROLLER
GYRO_LOCAL	3 × Float32 12 bytes	Data false / None Priority 0	SD_CARD
ACCEL_LOCAL	3 × Float32 12 bytes	Data false / None Priority 0	SD_CARD
IMU_DATA	6 × Float32 24 bytes	Data false / None Priority 0	GROUND_STATION
IMU_LOCAL	6 × Float32 24 bytes	Data false / None Priority 0	SD_CARD
BAROMETER_LOCAL	3 × Float32 12 bytes	Data false / None Priority 0	SD_CARD
ASCENT_LOCAL	6 × Float32 24 bytes	Data false / None Priority 0	SD_CARD
DESCENT_LOCAL	4 × Float32 16 bytes	Data false / None Priority 0	SD_CARD
AV_BAY_UNDERGLOW	1 × UInt8 1 bytes	Data true / Ordered Priority 16	AV_BAY_UNDERGLOW_OWNER
FLIGHT_BUZZER	1 × UInt8 1 bytes	Data true / Ordered Priority 16	FLIGHT_CONTROLLER
RESERVED_135	1 × Float32 4 bytes	Data true / Ordered Priority 16	None
RESERVED_136	1 × Float32 4 bytes	Data true / Ordered Priority 16	None
DAQ_LOADCELL_CAL IBRATION	4 × Float32 16 bytes	Data true / Ordered Priority 16	SD_CARD

A.3 Schema descriptions

Descriptions below preserve the board schema intent; units, enumerations and semantics absent from the schema need agreement with the owning board.

Type	Description
GENERIC_ERROR	Generic Error: Dynamic string payload (human-readable error message).
GPS_DATA	GPS Data: $3 \times \text{float32}$ elements (latitude, longitude, altitude).
GYRO_DATA	Gyro Data: $3 \times \text{float32}$ elements (x, y, z). Logged only to GroundStation.
ACCEL_DATA	Accel Data: $3 \times \text{float32}$ elements (x, y, z). Logged only to GroundStation.
BATTERY_VOLTAGE	Battery Voltage: $1 \times \text{float32}$ element.
BATTERY_CURRENT	Battery Current: $1 \times \text{float32}$ element.
BAROMETER_DATA	Barometer Data: $3 \times \text{float32}$ elements (alt, prs, tmp). Logged only to GroundStation.
ABORT	Abort: Dynamic string payload (reason / code / message).
FUEL_FLOW	Fuel Flow: $1 \times \text{float32}$ element.
VALVE_COMMAND	Valve Command: $1 \times \text{uint8}$ element (command).
FLIGHT_COMMAND	Flight Command: $1 \times \text{uint8}$ element (command).
FUEL_TANK_PRESSURE	Fuel Tank Pressure: $1 \times \text{float32}$ element.
FLIGHT_STATE	Flight State: $1 \times \text{uint8}$ element.
HEARTBEAT	Heartbeat: No payload.
WARNING	Warning: Dynamic string payload (human-readable warning text).
MESSAGE_DATA	Message Data: Dynamic string payload (free-form log message).
ACTUATOR_COMMAND	Commands to be sent to the actuator board (1 u8)
UMBILICAL_STATUS	The status of the umbilical setup (valves, actuator position, ingnighter state, etc)(2 u8's valve, position)
KG1000	1000kg loadcell values (1 float32)
KG50	50kg Loadcell (1 float32)
GPS_SATELLITE_NUMBER	(1 x U8) The number of satellites the gps sees in the sky.
EULER_ANGLES	Initial condition euler angles: $3 \times \text{float32}$ values.
ORDERED_MESSAGE	Ordered Message: MessageData variant for responses to manual commands and critical state changes.
ASCENT_STATE	Ascent State: Quaternion vector ($4 \times 1 \times \text{float32}$) and shared state ($2 \times 1 \times \text{float32}$). Logged only to GroundStation.
DESCENT_STATE	Descent State: low pass coordinates ($2 \times 1 \times \text{float32}$) and shared state ($2 \times 1 \times \text{float32}$). Logged only to GroundStation.
ASCENT_BIASES	Ascent Biases: pre-launch command to update biases of Ascent KF ($4 \times \text{float32}$).
GYRO_LOCAL	Gyro Data: $3 \times \text{float32}$ elements (x, y, z). Logged only to SD.
ACCEL_LOCAL	Accel Data: $3 \times \text{float32}$ elements (x, y, z). Logged only to SD.
IMU_DATA	Accel and Gyro Data: $6 \times \text{float32}$ elements (ax, ay, az, gx, gy, gz). Logged only to GroundStation.
IMU_LOCAL	Accel and Gyro Data: $6 \times \text{float32}$ elements (ax, ay, az, gx, gy, gz). Logged only to SD.
BAROMETER_LOCAL	Barometer Data: $3 \times \text{float32}$ elements (alt, prs, tmp). Logged only to SD.
ASCENT_LOCAL	Ascent State: Quaternion vector ($4 \times 1 \times \text{float32}$) and shared state ($2 \times 1 \times \text{float32}$). Logged only to SD.
DESCENT_LOCAL	Descent State: low pass coordinates ($2 \times 1 \times \text{float32}$) and shared state ($2 \times 1 \times \text{float32}$). Logged only to SD.
AV_BAY_UNDERGLOW	Persistent avionics-bay underglow network variable (0 off, 1 on).
FLIGHT_BUZZER	Persistent Flight Computer buzzer network variable (0 off, 1 on).
RESERVED_135	Reserved legacy rate ID; telemetry rates are compile-time board settings.
RESERVED_136	Reserved legacy rate ID; telemetry rates are compile-time board settings.
DAQ_LOADCELL_CALIBRATION	DAQ calibration [KG1000 slope, KG1000 intercept, IADC slope, IADC intercept].

A.4 Separate protocol contracts

Discovery, compact side transport, time synchronization and managed-variable control envelopes belong to SEDSNet. The OTA application protocol is separately defined in Section 8; it is not an extra entry in this telemetry-type list.